R1.01 - Initiation au Développement : Algorithmique, Variables et Structures de Contrôle

Thème principal : Les fondements de la programmation impérative : déclarations, typage, affectation, structures conditionnelles et boucles

Synthèse exécutive : Ce cours pose les bases indispensables du développement informatique en BUT Informatique. L'objectif est de comprendre comment structurer une pensée logique pour résoudre un problème algorithmique de façon rigoureuse. Nous couvrons la gestion des variables en mémoire (nom, type, valeur), les opérateurs logiques, les choix conditionnels (SI / SINON) ainsi que la répétition d'instructions grâce aux boucles (POUR, TANT QUE).

1. Les Variables et la Représentation en Mémoire

En programmation impérative, un ordinateur traite des données conservées temporairement dans sa mémoire vive. Pour accéder facilement à une case mémoire sans manipuler son adresse hexadécimale, on utilise une variable. Une variable se caractérise par : un identificateur (son nom), un type (définissant la plage de valeurs et l'espace alloué en octets), et une valeur. Avant toute utilisation, la variable doit être déclarée. L'opération d'affectation (notée '<-' en pseudo-code ou '=' dans la plupart des langages) évalue d'abord l'expression située à droite puis écrase la valeur précédente contenue dans la variable située à gauche.

- Variable : Emplacement nommé dans la mémoire vive (RAM) permettant de stocker une valeur modifiable au cours de l'exécution.
- Typage : Attribution d'un format spécifique (Entier, Réel, Chaîne, Booléen) déterminant la taille en mémoire et les opérations valides.
- Affectation : Action de remplacer la valeur d'une variable par le résultat d'une évaluation ou d'une constante.

Cas concret enseignant : *Le professeur compare la mémoire RAM à un grand meuble à tiroirs : déclarer une variable 'age : Entier', c'est coller une étiquette 'age' sur un tiroir spécifique et réserver l'espace nécessaire pour y glisser un nombre entier.*

2. Les Structures Conditionnelles (Prise de Décision)

Les structures conditionnelles permettent d'exécuter des blocs d'instructions différents en fonction de l'état du système ou de saisies utilisateur. Si l'expression testée est VRAIE, le bloc 'ALORS' est exécuté. Si elle est FAUSSE, le flux saute au bloc 'SINON' (s'il existe). Pour éviter des erreurs d'exécution ou des comportements imprévus, il est crucial d'étudier l'exhaustivité des cas possibles et de faire attention aux tests d'égalité.

- Condition Booléenne : Expression logique évaluable uniquement à VRAI (True) ou FAUX (False).
- Bloc SI / ALORS / SINON : Structure permettant de dériver l'exécution du code selon la valeur d'une condition.
- Opérateurs Logiques : ET, OU, NON permettant de combiner plusieurs conditions simples.

Cas concret enseignant : *Exemple de la borne du parking du campus de Bayonne : si la carte étudiant est valide ET que le parking n'est pas complet, la barrière se lève ; sinon un message d'erreur est affiché sur l'écran.*

3. Les Structures Répétitives (Boucles)

Les boucles évitent la duplication de code en répétant un même bloc d'instructions. Dans une boucle POUR, un compteur s'incrémente automatiquement à chaque passage jusqu'à atteindre la borne finale. Dans une boucle TANT QUE, la condition est évaluée avant chaque tour : il est donc impératif de modifier à l'intérieur de la boucle au moins une variable présente dans la condition pour garantir l'arrêt du programme.

- Boucle POUR (FOR) : Structure itérative utilisée lorsque le nombre exact de répétitions est connu à l'avance.

• Boucle TANT QUE (WHILE) : Structure itérative exécutée tant qu'une condition reste vraie, adaptée aux répétitions indéterminées.

• Boucle Infinie : Piège classique survenant lorsque la condition de sortie d'une boucle TANT QUE ne devient jamais fausse.

Cas concret enseignant : *Le professeur cite le bug du distributeur automatique de billets : si le système ne met pas à jour l'état du lecteur après le retrait de la carte, la boucle de saisie du code PIN tourne à l'infini et bloque le distributeur.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi doit-on obligatoirement spécifier le type d'une variable en C ou C++ alors que ce n'est pas nécessaire en Python ?
Réponse du professeur : En C ou C++, le typage est statique : le compilateur doit réserver une taille exacte de mémoire RAM dès la compilation (par exemple 4 octets pour un entier). En Python, le typage est dynamique : l'interpréteur devine et gère le type à l'exécution, ce qui apporte de la souplesse mais consomme plus de ressources et peut masquer des bugs de typage.

Question étudiant : Quelle est la différence entre l'opérateur '=' et l'opérateur '==' ?
Réponse du professeur : C'est une erreur classique qui coûte cher en partiel : '=' est l'opérateur d'affectation (il stocke une valeur dans une variable), alors que '==' est un opérateur de comparaison (il vérifie si deux valeurs sont égales et renvoie VRAI ou FAUX).

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme qui demande 5 notes (sur 20) à un utilisateur, calcule la moyenne arithmétique de ces notes, puis affiche 'Admis' si la moyenne est supérieure ou égale à 10, ou 'Ajourné' dans le cas contraire.

Résolution pas à pas :
1. Déclarer les variables nécessaires : 'somme' (Réel initialisé à 0.0), 'note' (Réel), 'moyenne' (Réel), et 'i' (Entier pour la boucle). 2. Utiliser une boucle POUR 'i' allant de 1 à 5 : a. Demander à l'utilisateur de saisir la note 'i'. b. Ajouter la note saisie à la variable 'somme' (somme <- somme + note). 3. Calculer la moyenne : 'moyenne <- somme / 5.0'. 4. Utiliser une condition SI : a. SI moyenne >= 10.0 ALORS afficher 'Admis'. b. SINON afficher 'Ajourné'.

Résultat : L'algorithme parcourt exactement 5 itérations, accumule les valeurs saisies de manière déterministe, calcule correctement la moyenne arithmétique et applique le test conditionnel pour délivrer le statut de l'étudiant.

■ Auto-Évaluation Flash d'Examen

Q : Que se passe-t-il si la condition d'une boucle TANT QUE est fausse dès le premier contrôle ?

R : Le corps de la boucle n'est jamais exécuté et le programme passe directement à l'instruction située après le bloc TANT QUE.

Q : Quelle est la règle pour choisir entre une boucle POUR et une boucle TANT QUE ?

R : On choisit la boucle POUR si le nombre de tours à effectuer est connu dès le départ ; on choisit TANT QUE si le nombre de répétitions dépend d'un événement ou d'une condition dynamique.

Q : Pourquoi est-il risqué de comparer deux nombres flottants (décinaux) directement avec '==' ?

R : À cause des approximations de codage en binaire de la norme IEEE 754, deux calculs identiques en théorie peuvent donner des valeurs minuscules de différence (ex: 0.1 + 0.2 != 0.3).